RESTAURANT LIST PROJECT GUIDE

데이터를 읽고
지도를 화면에 표시하는 방법

CSV · JavaScript 객체 · Kakao Maps · GeoJSON · 마커 · GPS · 길찾기

프로젝트에서 실제 사용한 코드와 작동 원리를
파일 위치·실행 순서·주석과 함께 정리한 설명서

목차

프로젝트 데이터 처리부터 지도 표시와 부가 기능까지 실제 실행 순서에 맞춰 정리했습니다.

장	내용	페이지
1	전체 구조를 먼저 이해하기	3
2	HTML에 지도 표시 공간 만들기	4
3	CSV 파일 경로를 안전하게 만드는 방법	5
4	fetch로 CSV 파일 읽기	6
5	CSV 문자열을 행과 열로 분리하기	7
6	CSV 행을 지도용 식당 객체로 바꾸기	8
7	CSV를 한 번만 읽도록 Promise 보관하기	9
8	서비스 기능을 다른 파일에 공개하기	9
9	카카오 지도 SDK를 화면에 불러오기	10
10	카카오 지도 객체 생성하기	11
11	GeoJSON으로 행정동 경계 표시하기	12
12	CSV 식당을 카카오 마커로 표시하기	13
13	지도·경계·CSV 마커를 동시에 준비하기	14
14	GPS와 길찾기가 지도에 연결되는 방법	15
15	자주 발생하는 오류와 확인 순서	16
16	카카오 지도 구현 요약	17
17	메인 화면의 Google Maps SDK와 지도	18
18	구글 음식점 마커와 화면 범위	19
19	구글 GPS 마커와 정확도 원	20
20	상세 지도와 카카오·구글 비교	21

읽는 순서
처음 보는 경우 1장부터 순서대로 읽고, 특정 기능만 확인하려면 CSV·지도 SDK·GeoJSON·마커·GPS 항목의 페이지로 바로 이동하면 됩니다.

1. 전체 구조를 먼저 이해하기

이 프로젝트의 지도는 CSV 파일을 지도에 직접 넣는 방식이 아니다. CSV 텍스트를 읽고 JavaScript 객체 배열로 변환한 다음, 각 객체의 위도와 경도로 카카오 마커를 만든다.

순서	담당 파일	하는 일
1	food_page.html	지도 영역과 JavaScript 파일을 준비
2	map-config-KakaoJS.js	카카오 JavaScript 키 제공
3	restaurantService.js	CSV 요청 · 파싱 · 식당 객체 변환
4	restaurantMap.js	SDK 로딩 · 지도 · 경계 · 마커 생성
5	geolocationService.js	GPS 현재 위치와 실시간 추적
6	directionsService.js	카카오 자동차 길찾기 REST API 호출

데이터 흐름

```
CSV 파일
  ↓ fetch()와 response.text()
CSV 문자열
  ↓ parseCsv()
행 객체 배열
  ↓ toRestaurant()
식당 객체 배열
  ↓ createRestaurantMarkers()
kakao.maps.Marker 객체
```

> 핵심
> 파일을 읽는 기술과 지도에 표시하는 기술은 분리되어 있다. RestaurantService가 데이터를 만들고 RestaurantMap이 화면에 그린다.

2. HTML에 지도 표시 공간 만들기

파일: html/food_page.html 79~80번째 줄

```
<!-- 카카오 지도가 들어갈 빈 HTML 영역이다. -->
<div id="map-area">
  <div id="map-api" aria-label="카카오 지도"></div>
</div>
```

지도 API는 새로운 웹페이지를 만드는 것이 아니라 지정한 div 내부에 지도 화면을 그린다. JavaScript는 getElementById('map-api')로 이 요소를 찾아 카카오 Map 생성자에 전달한다.

지도 영역에는 반드시 크기가 필요하다

```
#map-api {
  width: 100%;  /* 부모 영역의 가로 폭을 사용한다. */
  height: 100%; /* 또는 px, vh 등 실제 높이를 지정한다. */
}
```

div의 높이가 0이면 지도 객체가 정상 생성되어도 화면에는 보이지 않는다. 따라서 지도 문제를 확인할 때는 API 키뿐 아니라 HTML 요소와 CSS 높이도 함께 확인해야 한다.

JavaScript 파일 실행 순서

파일: html/food_page.html 157~167번째 줄

```
<!-- 1. 키와 데이터 서비스를 먼저 준비한다. -->
<script src="../js/map-config-KakaoJS.js"></script>
<script src="../js/map-config-KakaoRest.js"></script>
<script src="../js/restaurantService.js"></script>
<script src="../js/geolocationService.js"></script>

<!-- 2. 준비된 기능을 사용하는 지도 코드를 실행한다. -->
<script src="../js/restaurantMap.js"></script>
<script src="../js/directionsService.js"></script>
```

> ★ Q&A; 예상 질문과 답변
> 질문 restaurantMap.js를 API 키 파일이나 restaurantService.js보다 먼저 실행하면 왜 문제가 생기나요?
>
> 답변 restaurantMap.js가 실행될 때 필요한 window.KAKAO_JS_MAPS_API_KEY와 window.RestaurantService가 아직 만들어지지 않기 때문이다. 따라서 키와 데이터 서비스를 제공하는 파일을 먼저 실행해야 한다.

3. CSV 파일 경로를 안전하게 만드는 방법

파일: js/restaurantService.js 6~9번째 줄

```
// 현재 실행 중인 JS 파일 위치를 기준으로 CSV의 절대 주소를 만든다.
const RESTAURANT_CSV_URL = new URL(
  '../전국_음식점_정보csv/res_Food_List_geocoded.csv',
  document.currentScript.src
).href;
```

document.currentScript.src는 현재 실행 중인 restaurantService.js의 URL이다. new URL(상대경로, 기준URL)은 두 값을 합쳐 브라우저가 요청할 수 있는 완전한 주소를 만든다. .href는 완성된 주소 문자열을 꺼낸다.

왜 단순 상대경로보다 안전한가?

fetch('../파일.csv')처럼 작성하면 현재 HTML 페이지 주소를 기준으로 해석된다. 같은 JS 파일을 서로 다른 폴더의 HTML에서 사용하면 경로가 달라질 수 있다. currentScript를 기준으로 하면 JS 파일의 위치를 기준으로 항상 같은 CSV를 찾는다.

```
// HTML 주소가 기준이 되는 방식
fetch('../data/file.csv');

// restaurantService.js 주소가 기준이 되는 방식
const csvUrl = new URL('../data/file.csv', document.currentScript.src).href;
fetch(csvUrl);
```

> 주의
> 브라우저 보안 정책 때문에 HTML 파일을 file://로 직접 열면 fetch가 실패할 수 있다. VS Code Live Server 같은 로컬 웹 서버로 실행해야 한다.

4. fetch로 CSV 파일 읽기

파일: js/restaurantService.js 89~102번째 줄

```
async function loadRestaurants() {
  // CSV 파일을 HTTP로 요청하고 응답이 올 때까지 기다린다.
  const response = await fetch(RESTAURANT_CSV_URL, {
    cache: 'no-store' // 저장된 응답 대신 최신 파일을 요청한다.
  });

  // HTTP 상태가 200~299가 아니면 오류로 처리한다.
  if (!response.ok) {
    throw new Error(`CSV 요청 실패 (${response.status})`);
  }

  // CSV는 JSON이 아니므로 text()로 문자열을 읽는다.
  const csvText = await response.text();

  return parseCsv(csvText)
    .map(toRestaurant) // 각 행을 식당 객체로 변환한다.
    .filter(Boolean);  // null로 판정된 잘못된 행을 제거한다.
}
```

response.text()와 response.json()의 차이

메서드	사용 대상	결과
response.text()	CSV, HTML, 일반 텍스트	문자열
response.json()	JSON, GeoJSON	JavaScript 객체 또는 배열

★ Q&A; 예상 질문과 답변

질문 CSV 파일을 읽을 때 response.json()이 아니라 response.text()를 사용하는 이유는 무엇인가요?

답변 CSV는 JSON 문법이 아닌 일반 텍스트 형식이다. 먼저 response.text()로 문자열을 받은 다음 parseCsv()로 행과 열을 분리해야 한다. response.json()을 사용하면 JSON 형식 오류가 발생한다.

5. CSV 문자열을 행과 열로 분리하기

파일: js/restaurantService.js 14~58번째 줄

CSV 한 줄 파싱

```
function parseCsvLine(line) {
  const values = [];
  let value = '';
  let insideQuotes = false;

  for (let index = 0; index < line.length; index += 1) {
    const character = line[index];

    if (character === '"') {
      insideQuotes = !insideQuotes; // 따옴표 내부 상태를 바꾼다.
    } else if (character === ',' && !insideQuotes) {
      values.push(value.trim()); // 실제 열 구분 쉼표에서만 자른다.
      value = '';
    } else {
      value += character;
    }
  }

  values.push(value.trim());
  return values;
}
```

단순히 line.split(',')를 사용하면 "춘천시, 교동"처럼 따옴표 안에 포함된 쉼표까지 열 구분자로 잘못 인식한다. insideQuotes는 현재 문자가 따옴표 안에 있는지 기억한다.

전체 CSV를 객체 배열로 변환

```
const lines = csvText
  .replace(/^\uFEFF/, '')      // 파일 맨 앞의 BOM 문자를 제거한다.
  .split(/\r?\n/)          // Windows와 Unix 줄바꿈을 모두 처리한다.
  .filter(line => line.trim() !== ''); // 빈 줄을 제거한다.

const headers = parseCsvLine(lines[0]); // 첫 줄은 열 이름이다.

return lines.slice(1).map(line => {
  const values = parseCsvLine(line);
  return Object.fromEntries(
    headers.map((header, index) => [header.trim(), values[index] ?? ''])
  );
});
```

★ Q&A; 예상 질문과 답변

질문 CSV를 단순히 split(',')로 나누지 않고 별도의 parseCsvLine() 함수를 만든 이유는 무엇인가요?

답변 음식점명이나 주소가 따옴표로 감싸져 있고 그 안에 쉼표가 들어갈 수 있기 때문이다. parseCsvLine()은 따옴표 내부의 쉼표는 내용으로 유지하고 실제 열 구분 쉼표만 분리한다.

6. CSV 행을 지도용 식당 객체로 바꾸기

파일: js/restaurantService.js 61~87번째 줄

```
function toRestaurant(row, index) {
  // CSV 문자열 좌표를 실제 숫자로 변환한다.
  const latitude = Number(row['위도']);
  const longitude = Number(row['경도']);

  // 지오코딩 실패 또는 잘못된 좌표 행은 제외한다.
  if (row['지오코딩상태'] !== '성공' ||
    !Number.isFinite(latitude) ||
    !Number.isFinite(longitude)) {
    return null;
  }

  return {
    id: `restaurant-${index + 1}`,
    district: row['지역'],
    category: row['업종'],
    name: row['음식점명'],
    address: row['주소'] || row['변환주소'],
    latitude,
    longitude,
    rating: Number(row['평점']) || 0,
    reviewCount: Number(row['리뷰수']) || 0,
    imageUrl: row['이미지URL'] || ''
  };
}
```

CSV에서 읽은 모든 값은 처음에는 문자열이다. 위도·경도·평점·리뷰 수처럼 계산에 사용할 값은 Number()로 숫자로 바꾼다. Number.isFinite()는 좌표가 실제로 사용할 수 있는 유한한 숫자인지 검사한다.

변환 결과 예시

```
{
  id: 'restaurant-1',
  district: '교동',
  category: '한식',
  name: '별채식당',
  address: '강원특별자치도 춘천시 삭주로 75',
  latitude: 37.88,
  longitude: 127.73,
  rating: 4.1,
  reviewCount: 200
}
```

7. CSV를 한 번만 읽도록 Promise 보관하기

파일: js/restaurantService.js 10번째 줄, 104~114번째 줄

```
let restaurantsPromise = null;

function getRestaurants() {
  if (!restaurantsPromise) {
    // 최초 호출에서만 CSV 로딩 Promise를 저장한다.
    restaurantsPromise = loadRestaurants().catch(error => {
      restaurantsPromise = null; // 실패하면 다음 호출에서 재시도 가능
      throw error;
    });
  }

  // 이후 호출은 같은 Promise와 같은 결과를 공유한다.
  return restaurantsPromise;
}
```

지도, 주변 맛집, 상세 화면이 동시에 식당 데이터를 요청해도 매번 CSV를 다시 내려받지 않는다. Promise 자체를 저장하기 때문에 첫 요청이 아직 진행 중일 때 들어온 두 번째 호출도 같은 작업을 기다린다.

> ★ Q&A; 예상 질문과 답변
> 질문 완성된 식당 배열이 아니라 restaurantsPromise를 저장한 이유는 무엇인가요?
>
> 답변 첫 CSV 요청이 아직 끝나기 전에 여러 기능이 데이터를 요청해도 같은 로딩 작업을 함께 기다리게 하기 위해서다. 배열만 나중에 저장하면 로딩 중 들어온 호출이 CSV 요청을 중복 실행할 수 있다.

8. 서비스 기능을 다른 파일에 공개하기

```
// IIFE 내부 함수 중 외부에서 필요한 기능만 공개한다.
window.RestaurantService = {
  getRestaurants,
  findRestaurant
};
```

restaurantService.js는 즉시 실행 함수로 내부를 감싼다. 따라서 parseCsvLine 같은 내부 함수는 다른 파일에서 직접 호출할 수 없고, window.RestaurantService에 넣은 공개 함수만 사용할 수 있다.

9. 카카오 지도 SDK를 화면에 불러오기

파일: js/restaurantMap.js 256~272번째 줄

```
function loadKakaoMaps() {
  // 별도 설정 파일에 저장된 JavaScript 키를 가져온다.
  const apiKey = window.KAKAO_JS_MAPS_API_KEY?.trim();

  // 키 형식이 잘못되면 지도를 로딩하지 않는다.
  if (!/^[0-9a-f]{32}$/i.test(apiKey || '')) return;

  // SDK 다운로드 후 kakao.maps.load()로 지도 초기화를 시작한다.
  const start = () => kakao.maps.load(initFoodMap);
  if (window.kakao?.maps) return start();

  // 페이지에 SDK script 태그를 동적으로 추가한다.
  const script = document.createElement('script');
  script.src = `https://dapi.kakao.com/v2/maps/sdk.js?appkey=${encodeURIComponent(apiKey)}&autoload=false`;
  script.async = true;
  script.onload = start;
  script.onerror = () => fail('지도 SDK 로딩 실패');
  document.head.appendChild(script);
}
```

createElement('script')로 태그를 만들고 src에 SDK 주소를 넣는다. appendChild()로 document.head에 붙이면 브라우저가 SDK를 다운로드한다. autoload=false이므로 다운로드 후 kakao.maps.load()를 호출해야 초기화된다.

> 보안 구분
> 카카오 JavaScript 키는 지도 SDK 로딩에 사용한다. 자동차 길찾기 요청은 별도의 REST API 키를 Authorization 헤더에 사용한다.

10. 카카오 지도 객체 생성하기

파일: js/restaurantMap.js 229~233번째 줄

```
async function initFoodMap() {
  // HTML에서 지도를 그릴 div를 가져온다.
  const mapElement = document.getElementById('map-api');

  // 카카오 지도 객체를 생성해 div 내부에 표시한다.
  map = new kakao.maps.Map(mapElement, {
    center: new kakao.maps.LatLng(37.8813, 127.7298),
    level: 7
  });

  // 다른 기능에 지도 준비 완료 사실을 알린다.
  window.foodMap = map;
  document.dispatchEvent(new CustomEvent('restaurant-map:ready'));
}
```

각 코드의 역할

코드	역할
kakao.maps.Map	지정한 HTML 요소에 지도 본체 생성
kakao.maps.LatLng	위도·경도를 카카오 좌표 객체로 변환
center	처음 화면의 중심 좌표
level	지도 확대 단계 - 값이 작을수록 확대
CustomEvent	다른 파일에 지도 준비 완료 알림

★ Q&A; 예상 질문과 답변
질문 지도 SDK를 불러오는 것과 new kakao.maps.Map()으로 지도 객체를 만드는 것은 어떻게 다른가요?

답변 SDK 로딩은 kakao.maps.Map 같은 지도 기능의 설계도와 프로그램을 브라우저에 준비하는 과정이다. new kakao.maps.Map()은 준비된 SDK를 사용해 특정 HTML 요소 안에 실제 지도 인스턴스를 만드는 과정이다.

11. GeoJSON으로 행정동 경계 표시하기

파일: js/restaurantMap.js 164~170번째 줄

```
async function loadBoundaries() {
  // 시 경계와 행정동 경계를 동시에 요청한다.
  const responses = await Promise.all([
    fetch(DISTRICT_URL),
    fetch(CITY_URL)
  ]);

  // 둘 중 하나라도 실패하면 경계 표시를 중단한다.
  if (responses.some(response => !response.ok)) {
    throw new Error('경계 파일 로딩 실패');
  }

  // GeoJSON은 JSON 형식이므로 json()으로 변환한다.
  [districtGeoJson, cityGeoJson] = await Promise.all(
    responses.map(response => response.json())
  );
}
```

GeoJSON 좌표와 카카오 좌표의 순서

```
// GeoJSON: [경도, 위도]
// Kakao LatLng 생성자: (위도, 경도)
const latLng = coordinate =>
  new kakao.maps.LatLng(coordinate[1], coordinate[0]);
```

GeoJSON의 Polygon 또는 MultiPolygon 좌표를 반복해 kakao.maps.Polygon으로 만들면 행정동 면을 표시할 수 있다. 시 외곽선은 kakao.maps.Polyline으로 표시한다.

> ★ Q&A; 예상 질문과 답변
> 질문 GeoJSON 좌표를 LatLng로 바꿀 때 coordinate[1]과 coordinate[0]의 순서를 뒤집는 이유는 무엇인가요?
>
> 답변 GeoJSON 좌표는 [경도, 위도] 순서지만 kakao.maps.LatLng()는 (위도, 경도) 순서로 값을 받기 때문이다. 순서를 바꾸지 않으면 경계가 엉뚱한 위치에 표시된다.

12. CSV 식당을 카카오 마커로 표시하기

파일: js/restaurantMap.js 203~227번째 줄

```
function createRestaurantMarkers(restaurants) {
  // 이전 마커를 제거해 중복 표시를 막는다.
  restaurantMarkers.forEach(({ marker }) => marker.setMap(null));
  restaurantMarkers = [];

  restaurants.forEach(restaurant => {
    // CSV의 위도와 경도를 카카오 좌표 객체로 바꾼다.
    const position = new kakao.maps.LatLng(
      restaurant.latitude,
      restaurant.longitude
    );

    // 카카오 지도 전용 마커 객체를 생성한다.
    const marker = new kakao.maps.Marker({
      map,
      position,
      title: restaurant.name
    });

    // 마커 클릭 시 음식점 정보창을 연다.
    kakao.maps.event.addListener(marker, 'click', () => {
      openRestaurantGroup(marker, [restaurant]);
    });

    restaurantMarkers.push({ restaurants: [restaurant], marker });
  });
}
```

현재 실제 프로젝트는 같은 좌표의 음식점을 Map 객체로 먼저 묶어 핀 하나로 표시한다. 위 코드는 전체 흐름을 보기 쉽게 단순화한 예시이며 핵심 API는 실제 코드와 동일하다.

> 구글 마커와의 차이
> 카카오 지도에서는 kakao.maps.Marker를 사용한다. 메인 화면의 구글 지도에서 사용하는 google.maps.Marker와는 서로 다른 SDK 객체이며 호환되지 않는다.

13. 지도·경계·CSV 마커를 동시에 준비하기

파일: js/restaurantMap.js 246~251번째 줄

```
try {
  // 경계 로딩과 CSV 로딩·마커 생성을 동시에 시작한다.
  await Promise.all([
    loadBoundaries(),
    window.RestaurantService
      .getRestaurants()
      .then(createRestaurantMarkers)
  ]);

  // 모든 작업이 성공한 후 로딩 화면을 숨긴다.
  loader.hidden = true;
  controls.hidden = false;
} catch (error) {
  // 어느 한 작업이라도 실패하면 오류를 표시한다.
  fail(error.message);
}
```

Promise.all()을 사용하면 GeoJSON 경계 파일과 CSV 식당 파일을 순차적으로 기다리지 않고 동시에 불러온다. getRestaurants()가 반환한 식당 배열은 then(createRestaurantMarkers)로 마커 생성 함수에 전달된다.

실행 시점의 전체 순서

단계	완료 조건
SDK	kakao.maps.load() 완료
지도 본체	new kakao.maps.Map() 실행
경계	GeoJSON 두 파일 로딩·변환 완료
마커	CSV 로딩·파싱·좌표 변환 완료
화면 활성화	경계와 마커 작업이 모두 성공

14. GPS와 길찾기가 지도에 연결되는 방법

GPS 현재 위치

```
// 위치가 갱신되면 일반 좌표를 카카오 좌표로 변환한다.
const positionLatLng = new kakao.maps.LatLng(
  position.latitude,
  position.longitude
);

// 기존 현재 위치 표시가 있으면 위치만 갱신한다.
currentLocationMarker.setPosition(positionLatLng);
```

GeolocationService의 watchPosition()이 새 위치를 전달하면 RestaurantMap이 현재 위치 표시를 이동한다. enableHighAccuracy, timeout, maximumAge는 브라우저 위치 요청 조건이며 별도의 수학적 좌표 보정 알고리즘은 아니다.

카카오 자동차 길찾기

```
// REST API가 이해하는 출발·도착 쿼리를 만든다.
const query = new URLSearchParams({
  origin: `${origin.longitude},${origin.latitude}`,
  destination: `${destination.longitude},${destination.latitude}`
});

// REST API 키를 인증 헤더에 넣어 경로를 요청한다.
const response = await fetch(`${DIRECTIONS_ENDPOINT}?${query}`, {
  headers: { Authorization: `KakaoAK ${getApiKey()}` }
});
```

길찾기 REST API의 결과 좌표를 kakao.maps.LatLng 배열로 변환하고 kakao.maps.Polyline으로 연결하면 지도에 자동차 경로선이 표시된다.

15. 자주 발생하는 오류와 확인 순서

증상	가능한 원인	확인할 내용
CSV 요청 실패	상대경로·file:// 실행	로컬 서버와 Network 탭 확인
한글 첫 열 이름 오류	CSV BOM	replace(/^\uFEFF/, '') 적용
일부 열이 밀림	따옴표 안 쉼표	parseCsvLine의 insideQuotes 확인
지도가 안 보임	div 높이 0·키 오류	CSS 크기와 SDK 요청 확인
마커가 안 보임	좌표 문자열·순서 오류	Number 변환과 위도·경도 확인
마커가 중복됨	기존 마커 미제거	marker.setMap(null) 후 배열 초기화
경계가 뒤집힘	GeoJSON 좌표 순서	[경도, 위도]를 LatLng(위도, 경도)로 변환
길찾기 401 오류	REST 키·헤더 오류	Authorization: KakaoAK ... 확인

개발자 도구에서 확인하는 순서

1. Console: JavaScript 오류 메시지 확인
2. Network: CSV·GeoJSON·SDK 요청의 상태 코드 확인
3. Elements: #map-api 요소와 실제 높이 확인
4. Application/보안 설정: API 키의 허용 도메인 확인
5. 데이터: CSV 위도·경도가 유효한 숫자인지 확인

> 중요
> API 키는 종류가 다르다. 카카오 지도 화면은 JavaScript 키, 자동차 길찾기 REST 요청은 REST API 키를 사용한다.

16. 카카오 지도 구현 요약

이 프로젝트에서 CSV와 지도가 연결되는 핵심은 다음 네 문장으로 정리할 수 있다.

```
// 1. CSV를 문자열로 읽는다.
const csvText = await (await fetch(CSV_URL)).text();

// 2. 문자열을 식당 객체 배열로 바꾼다.
const restaurants = parseCsv(csvText).map(toRestaurant).filter(Boolean);

// 3. 카카오 지도 본체를 만든다.
const map = new kakao.maps.Map(mapElement, mapOptions);

// 4. 각 식당 좌표로 카카오 마커를 만든다.
restaurants.forEach(restaurant => {
  new kakao.maps.Marker({
    map,
    position: new kakao.maps.LatLng(
      restaurant.latitude,
      restaurant.longitude
    )
  });
});
```

질문	핵심 답변
CSV는 어떻게 읽나?	fetch 후 response.text() 사용
CSV를 왜 파싱하나?	문자열을 열 이름이 있는 객체 배열로 변환하기 위해
지도는 어디에 생기나?	HTML의 #map-api 요소 안에 생성
마커는 무엇인가?	kakao.maps.Marker로 만든 카카오 SDK 객체
경계는 어떻게 표시하나?	GeoJSON 좌표를 Polygon · Polyline으로 변환
왜 Promise.all을 쓰나?	CSV와 GeoJSON을 동시에 불러오기 위해

기준 소스: html/food_page.html, js/restaurantService.js, js/restaurantMap.js, js/geolocationService.js, js/directionsService.js

17. 메인 화면의 Google Maps SDK와 지도

메인 홈페이지의 작은 지도는 카카오 지도가 아니라 Google Maps JavaScript API를 사용한다. 담당 파일은 index.html과 js/restaurant_homepageMap.js이다.

Google Maps SDK 동적 로딩

```
function loadGoogleMaps() {
  // 별도 설정 파일의 구글 API 키를 읽는다.
  const apiKey = window.GOOGLE_MAPS_API_KEY?.trim();
  const valid = /^AIza[0-9A-Za-z_-]{30,}$/.test(apiKey || '');
  if (!valid) return;

  // SDK가 호출할 전역 콜백 함수를 등록한다.
  window.initHomepageMap = initHomepageMap;
  window.gm_authFailure = () => showGoogleMapError();

  // callback 파라미터가 SDK 로딩 후 지도 초기화를 실행한다.
  const script = document.createElement('script');
  script.src =
    'https://maps.googleapis.com/maps/api/js' +
    `?key=${encodeURIComponent(apiKey)}` +
    '&callback=initHomepageMap&v=weekly' +
    '&language=ko&region=KR';
  script.async = true;
  script.defer = true;
  script.onerror = () => showGoogleMapError();
  document.head.appendChild(script);
}
```

메인 화면 지도 객체 생성

```
function initHomepageMap() {
  const mapElement = document.getElementById('homepage-map');

  // Google 지도 본체를 HTML 요소 안에 생성한다.
  map = new google.maps.Map(mapElement, {
    center: { lat: 37.8813, lng: 127.7298 },
    zoom: 13,
    mapTypeControl: false,
    streetViewControl: false,
    fullscreenControl: false
  });
}
```

카카오는 center에 kakao.maps.LatLng 객체를 전달하지만, 구글 지도는 { lat, lng } 형태의 일반 객체도 좌표로 받을 수 있다. 카카오는 level, 구글은 zoom으로 확대 단계를 설정한다.

> ★ Q&A; 예상 질문과 답변
> 질문 메인 화면 지도와 맛집 탐색 화면 지도는 같은 지도 API인가요?
>
> 답변 아니다. 메인 화면의 #homepage-map은 google.maps.Map을 사용하고, 맛집 탐색 화면의 #map-api는 kakao.maps.Map을 사용한다. 각 SDK의 지도·마커 객체는 서로 호환되지 않는다.

18. 구글 음식점 마커와 화면 범위

파일: js/restaurant_homepageMap.js 140~219번째 줄

```
// 기존 구글 마커를 제거한다.
restaurantMarkers.forEach(({ marker }) => marker.setMap(null));
restaurantMarkers = [];

const bounds = new google.maps.LatLngBounds();

requestedRestaurants.forEach((restaurant, index) => {
  const position = coordinates.get(normalizeName(restaurant.name));
  if (!position) return;

  // 순위 번호를 표시하는 구글 지도 전용 마커다.
  const marker = new google.maps.Marker({
    map,
    position,
    label: {
      text: String(index + 1),
      color: '#ffffff',
      fontWeight: '700'
    },
    title: restaurant.name
  });

  marker.addListener('click', () => focusRestaurant(key));
  restaurantMarkers.push({ marker, key });
  bounds.extend(position); // 표시 범위에 마커 위치를 추가한다.
});

map.fitBounds(bounds, 64); // 모든 마커가 보이도록 지도를 맞춘다.
```

메인 화면 순위가 바뀌면 기존 마커를 제거하고 새 순위 음식점의 마커를 다시 만든다. LatLngBounds에 각 위치를 누적한 뒤 fitBounds()를 호출하면 여러 마커가 한 화면에 들어온다.

> ★ Q&A; 예상 질문과 답변
> 질문 카카오와 구글 모두 marker.setMap(null)을 사용하는데 같은 마커인가요?
>
> 답변 아니다. 메서드 이름과 사용 방식이 비슷할 뿐이다. 카카오는 kakao.maps.Marker, 구글은 google.maps.Marker로 서로 다른 SDK가 만든 객체다.

19. 구글 GPS 마커와 정확도 원

파일: js/restaurant_homepageMap.js 252~301번째 줄

```
const location = {
    lat: position.latitude,
    lng: position.longitude
};

// 현재 위치를 파란 원 모양의 구글 마커로 표시한다.
currentLocationMarker = new google.maps.Marker({
    map,
    position: location,
    title: '내 현재 위치',
    icon: {
        path: google.maps.SymbolPath.CIRCLE,
        fillColor: '#2563eb',
        strokeColor: '#ffffff',
        strokeWeight: 4,
        scale: 9
    }
});

// 브라우저가 제공한 예상 오차 반경을 지도 원으로 표시한다.
currentLocationAccuracyCircle = new google.maps.Circle({
    map,
    center: location,
    radius: position.accuracy,
    fillColor: '#2563eb',
    fillOpacity: 0.12,
    strokeOpacity: 0.35
});
```

position.accuracy는 미터 단위의 예상 오차 반경이다. google.maps.Circle의 radius도 미터 단위를 사용하므로 값을 그대로 전달해 사용자의 실제 위치가 있을 가능성이 높은 범위를 시각화한다.

> 중요
> 정확도 원은 GPS 좌표를 수정하는 보정 알고리즘이 아니다. 브라우저가 알려준 오차 범위를 지도에 보여주는 기능이다.

> ★ Q&A; 예상 질문과 답변
> 질문 파란 정확도 원이 클수록 GPS가 더 정확한 것인가요?
>
> 답변 반대다. accuracy 값과 원의 반지름이 클수록 예상 오차 범위가 넓으므로 정확도가 낮다. 원이 작을수록 현재 위치를 더 좁은 범위로 추정했다는 뜻이다.

20. 상세 지도와 카카오·구글 비교

음식점 상세 페이지의 구글 지도

파일: js/restaurantDetailMap.js 40~69번째 줄

```
const position = {
  lat: Number(restaurant.latitude),
  lng: Number(restaurant.longitude)
};

// 선택한 음식점 한 곳을 중심으로 상세 지도를 만든다.
map = new google.maps.Map(mapElement, {
  center: position,
  zoom: 17,
  mapTypeControl: false,
  streetViewControl: false,
  clickableIcons: false
});

// 상세 페이지에는 선택한 음식점 마커 하나만 표시한다.
marker = new google.maps.Marker({
  map,
  position,
  title: restaurant.name
});
```

프로젝트의 두 지도 비교

구분	카카오 지도	구글 지도
사용 화면	맛집 탐색·길찾기	메인 미니맵·상세 페이지
지도 객체	kakao.maps.Map	google.maps.Map
마커 객체	kakao.maps.Marker	google.maps.Marker
좌표	new kakao.maps.LatLng(위도, 경도)	{ lat: 위도, lng: 경도 }
확대 값	level - 작을수록 확대	zoom - 클수록 확대
화면 맞춤	map.setBounds()	map.fitBounds()
주요 특화 기능	GeoJSON 경계·자동차 길찾기	순위 마커·GPS 정확도 원

★ Q&A; 예상 질문과 답변

질문 왜 프로젝트에서 카카오 지도와 구글 지도를 함께 사용하나요?

답변 현재 구현은 화면 목적에 따라 나뉘어 있다. 맛집 탐색은 카카오의 행정동 경계와 길찾기를 사용하고, 메인·상세 화면은 구글 미니맵과 순위 마커를 사용한다. 기능을 합치려면 한 SDK 기준으로 별도 리팩터링이 필요하다.

추가 기준 소스: index.html, js/map-config.js, js/restaurant_homepageMap.js, js/restaurantDetailMap.js